

ElectronBot — Roadmap Revisado (Webcam-follow · Zephyr · SW PC)

Revisão técnica baseada em leitura direta do código (3.Software/SDK , _Tools/AHK-ExpansionPack). Documento de planejamento — nenhum código foi executado/alterado.

⚠ Correções de fato (vindas do código)

#	Suposição anterior	Realidade no código
C1	"Dá pra mover o servo sem mandar imagem"	Falso. O único caminho de envio é <code>Sync()</code> , que sempre faz upload do frame inteiro (172.800 B) (<code>SyncTask : 4×(84×512 + cauda 224)</code>). As 6 juntas viajam nos 32 bytes da cauda . ⇒ taxa do loop de controle = taxa de upload de frame inteiro; o display é sempre alimentado.
C2	Ler ângulos é direto	<code>GetJointAngles</code> reflete só o último sync , e o wrapper lê 2× com <code>sleep 10</code> no meio. Modo "physical-priority" custa um ciclo + handshake.
C3	Limites genéricos	Confirmado: raw <code>SetJointAngles(j1..j6)</code> → j1=cabeça (±15°) , j2=braço-esq-abrir, j3=braço-dir-levantar, j4=braço-dir-abrir, j5=braço-esq-levantar, j6=cintura/giro (±90°) . A webcam gira o j6 .
C4	"Câmera espelhada"	A bridge faz <code>flip(img, -1)</code> = rotação de 180° (câmera montada de cabeça pra baixo). A detecção e o sinal de controle têm que desfazer isso.
C5	—	<code>USB_ScanDevice(PID, VID)</code> tem a ordem dos args trocada ; VID/PID = <code>0x1001/0x8023</code> .

Impacto de C1: cada tick de controle empurra um frame inteiro. Não é gargalo de latência (upload assíncrono num worker, com `join` do anterior), mas implica **sempre definir a imagem do display** — decidir já qual cara mostrar (ex.: olhos estáticos) e a taxa-alvo (~20–30 Hz).

🕒 Lacunas identificadas

Fundação (faltava inteira)

- **Sem git.** Antes de tocar em firmware/SW: `git init` + **backup do .hex bom** (6.Tests/_Released) para rollback.
- **Spec escrita do protocolo USB** (framing 512/224 + bloco de 32 B). Hoje só existe implícita no código — **pré-requisito** do port Zephyr.

- **Build C++ reprodutível** (OpenCV 3.4.8 *debug* + paths `D:\Libraries\...`). Só importa se compilar; protótipo Python dispensa.

Setup / pré-requisitos (bloqueante)

- **Zadig / binding do driver** no VID/PID certo — o `.inf` do BotDriver aponta pro bootloader. Sem isso, nada conecta.
- **Calibração ServoToolKit**: IDs I2C (2,4,6,8,10,12), zero, PID, limites — **um servo por vez**; exige flashar o firmware do toolkit e depois voltar o normal.
- **Orçamento de energia**: 6 servos + display num barramento USB → risco de *brownout*. Limitar movimento simultâneo.

Frente 1 (webcam)

- Detector + **tracker**; política de "qual rosto" (maior/mais próximo / lock no dono).
- Malha: **deadzone**, suavização (EMA), **rate-limit**, cuidado com **cascata** (PID do servo embaixo).
- Comportamento **sem rosto** (segurar/centro/parque).
- Vertical = só **crop digital** (re-centra na saída).
- Saída: **OBS Virtual Camera** (via `pyvirtualcam`); medir latência fim-a-fim.
- Hot-plug/reconexão; modo **privacidade/parque**; câmera **0.3MP** (qualidade limitada).

Frente 2 (SW PC)

- **Decisão**: Python via **DLL do AHK** (pronta, C-ABI) ✓ — vs wrapper próprio compilado. Usar **as DLLs do pacote AHK** (só elas exportam `AHK_*`).
- **Windows-only** (libusb-win32). Linux/Mac = porta nova.
- **Licença** (provável GPL) se for distribuir.

Frente 3 (Zephyr)

- Escolher **stack USB do Zephyr** (device legado vs `usb_device_next` experimental) — risco; é onde mora o protocolo custom.
- GC9A01 **tem driver Zephyr** (`gc9x01x`), mas o modelo é **streaming custom por DMA** → provável bypass do subsistema de display.
- **Flash/recovery**: ST-Link + bootloader **DFU** (0483:5740).
- **Testes** (Ztest/Twister); manter os **2 firmwares coexistindo**; critério inegociável = **byte-compatível com o SDK do PC**.
- **ROI honesto**: a cabeça é quase um bridge USB↔SPI/I2C; o ganho do Zephyr é **manutenibilidade/ecossistema, não função**. Servo F042 (6KB RAM) **fica de fora**.

Transversal

Critérios de aceite por fase (DoD), registro de riscos, métrica de latência, e definir o **objetivo** (uso pessoal × distribuir).



Roadmap revisado

Fase	Entrega	Definition of Done
-1 · Fundação	<code>git init</code> + backup do <code>.hex</code> + spec do protocolo (1 pág) + [build env, se compilar]	Repo versionado, <code>.hex</code> guardado, spec escrita
0 · Bring-up & setup	Zadig + BotDriver; ServoToolKit (IDs/zero/limites/PID); rodar <code>Sample.exe</code>	"Robot connected!" + <code>happy.mp4</code> na cara e responde a <code>SetJointAngles</code>
1 · Webcam-follow (Python)	câmera→rosto→tracker→ P/PI no j6 (± 90, deadzone/EMA/rate-limit) , desfaz flip 180°, olhos estáticos no display, sem-rosto→centro; depois virtual cam	Rosto centrado no eixo horizontal; saída usável no Zoom/Teams
2 · Zephyr (cabeça)	spec → PoC <code>usbd</code> reproduz o protocolo (SDK do PC conecta) → display SPI/DMA → I2C servos → paridade	<code>Sample.exe</code> funciona igual contra o firmware Zephyr
3 · Servos (opcional)	só se trocar o MCU; senão manter o build LL	—

Registro de riscos

Risco	Sev.	Quando	Mitigação
Binding do driver / Zadig	Alto	Fase 0	Validar conexão antes de qualquer código novo
Brownout com 6 servos no USB	Médio	Fase 0/1	Limitar movimento simultâneo; medir corrente; só mover j6 na Fase 1
Reimplementar USB custom no Zephyr byte-a-byte	Alto	Fase 2	Spec primeiro; testar contra <code>Sample.exe</code> a cada passo
Oscilação da malha em cascata	Médio	Fase 1	EMA + deadzone + rate-limit; controle de velocidade
ROI do Zephyr baixo na função	Médio	Fase 2	Decidir se o objetivo é aprendizado/manutenção antes de investir

Sequência recomendada: Fase -1 → 0 → 1 (resultado visível, risco baixo, zero firmware) antes de encarar a Fase 2.